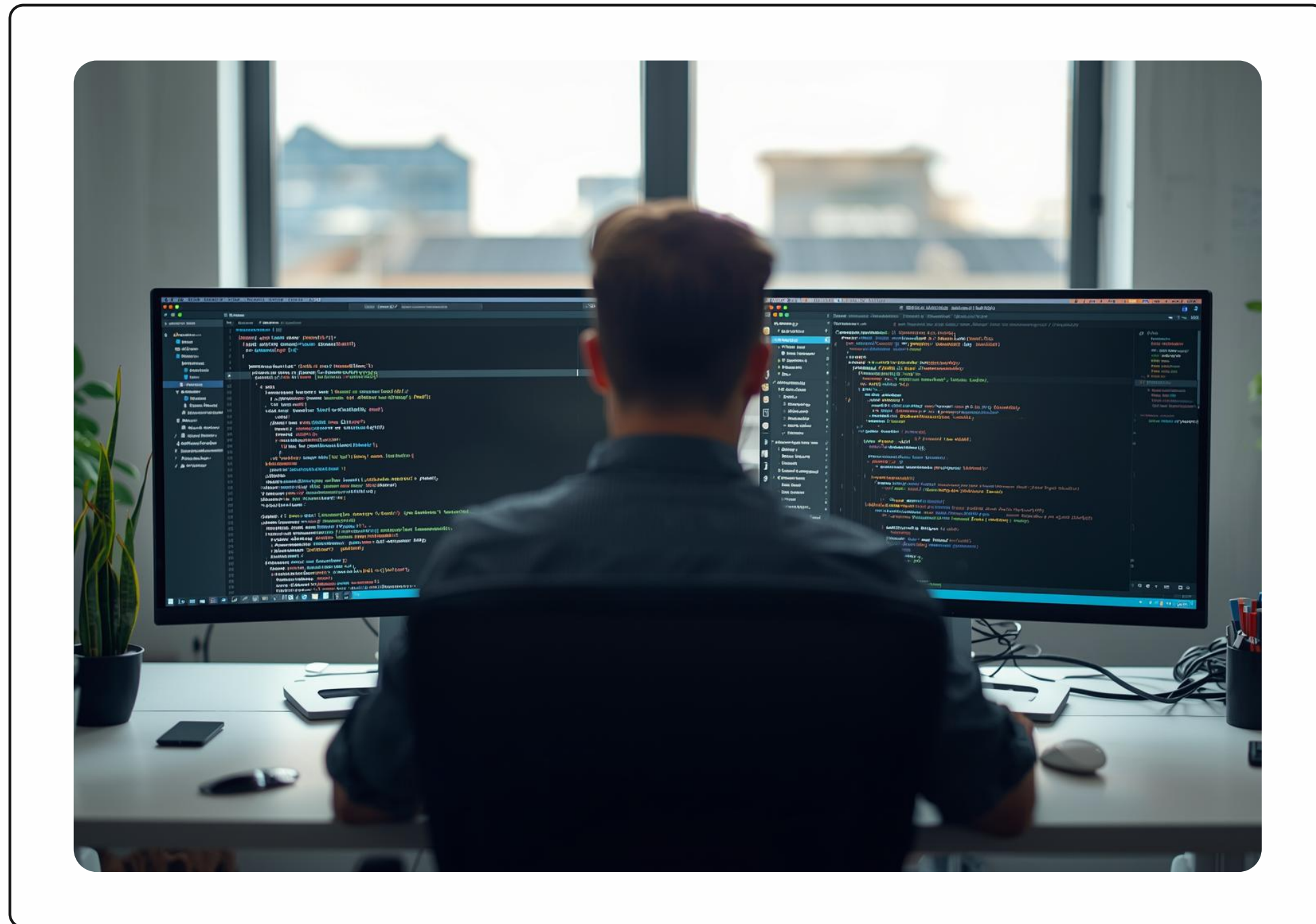


NPM (NODE PACKAGE MANAGER) COMPLETE CRASH COURSE

Installation,
Commands,
package.json,
Dependencies,
Versioning, and NPM
Scripts



WHAT IS NPM?

Node Package Manager

Package Manager

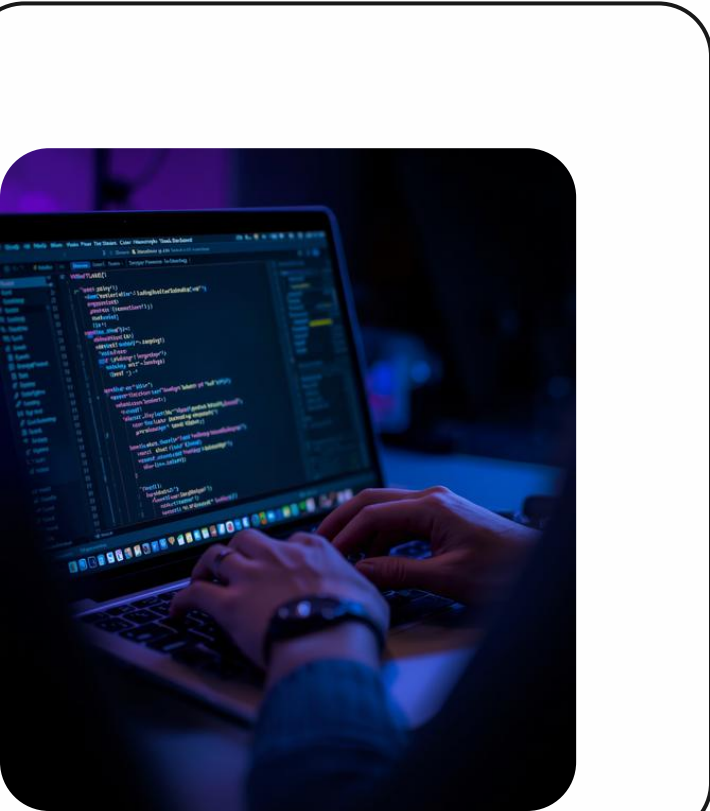
JavaScript package manager similar to Ruby Gems or Python PIP, installed automatically with Node.js.

Code Sharing

Makes sharing and reusing code incredibly easy across projects.

Packages

Access thousands of modules like Bootstrap, jQuery, Lodash, and more from npmjs.com.





Your Complete NPM Learning Path

Package Management

Installing, removing, updating, and listing packages.

package.json

Understanding the manifest file and project configuration.

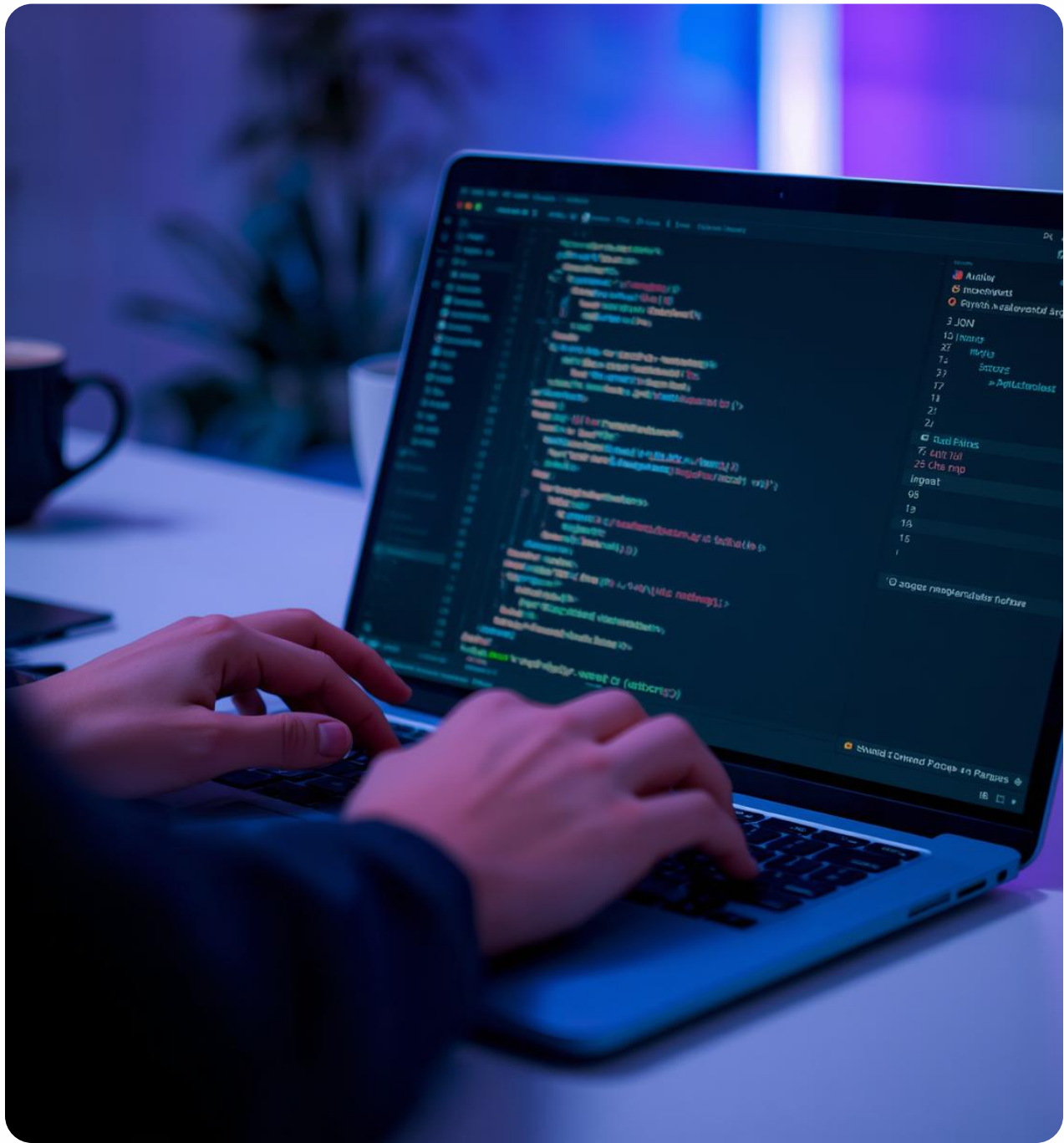
Dependencies

Local vs global, dev vs production dependencies.

Scripts & Versioning

NPM scripts automation and semantic versioning.

WHAT YOU'LL LEARN



Setting Up NPM

Install Node.js from nodejs.org (NPM comes bundled with it). Check your NPM version using `npm -v` or `npm -version`. Use `npm help` or just `npm` to see available commands. Visit npmjs.com to search and discover packages for your projects.

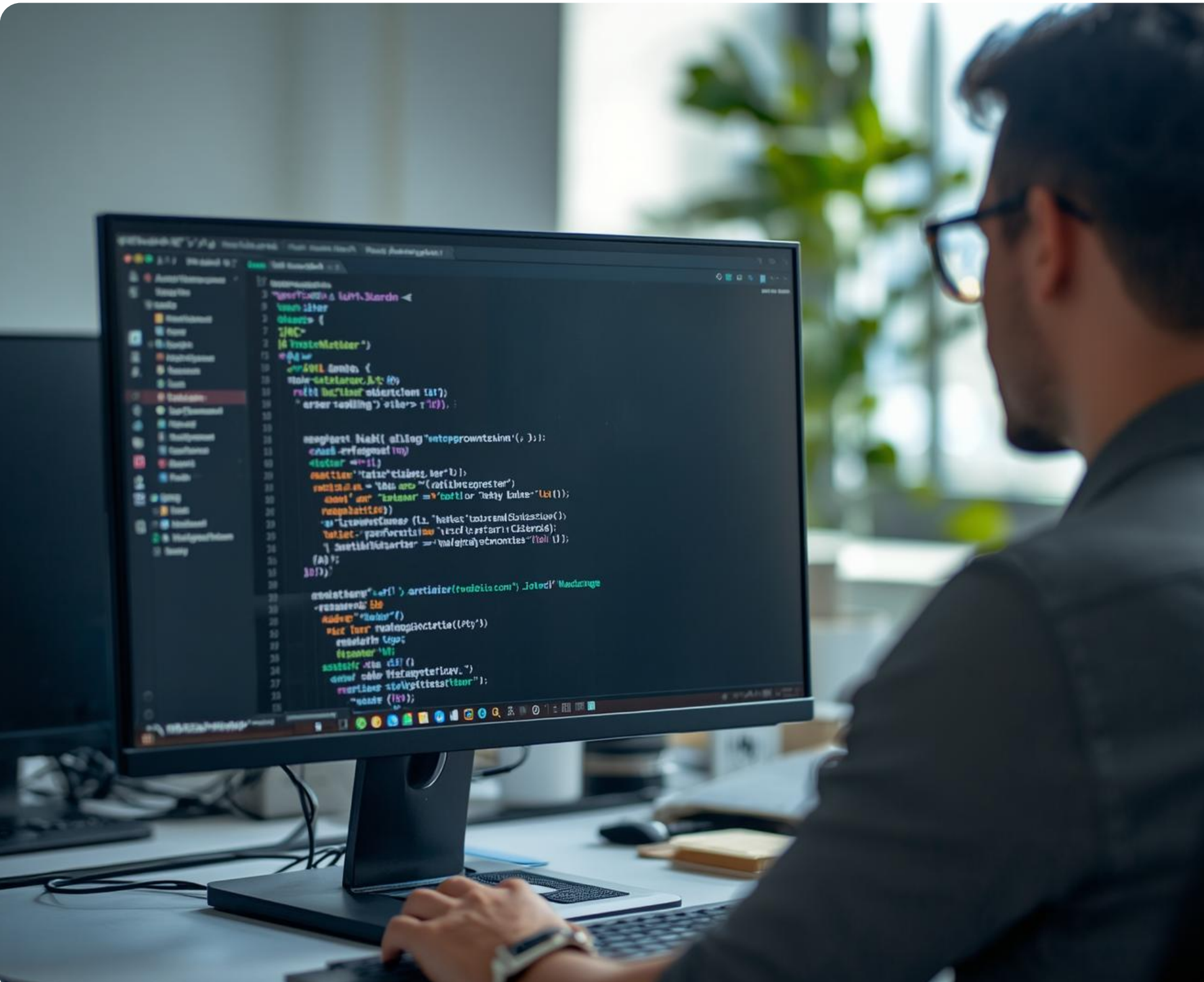
GETTING STARTED



UNDERSTANDING PACKAGE.JSON

The Heart of Any Node.js Project

The package.json file is the manifest that holds all essential app information: name, version, author, and license. It lists all dependencies with their specific versions, ensuring consistent installations across environments. This file also defines NPM scripts for automating tasks. Create it easily using the npm init command, which walks you through setup interactively.



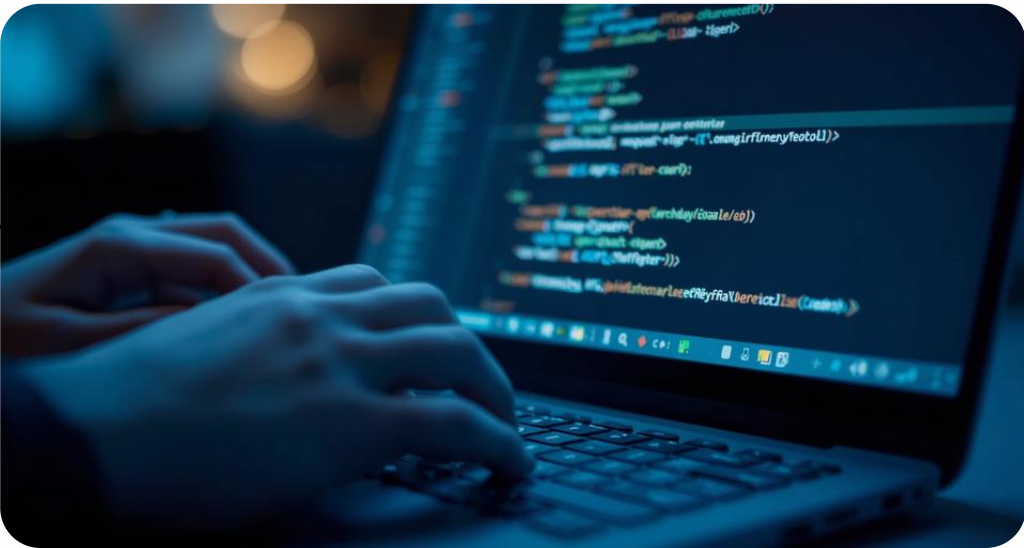


Creating package.json with npm init

npm init: interactive questionnaire to create package.json step by step. npm init -y or npm init --yes: accept all defaults instantly for quick setup.

Customize defaults with npm config set init-author-name "Your Name". Check with npm get init-author-name. Delete with npm config delete.

NPM INIT



INSTALLING PACKAGES LOCALLY

Adding Dependencies to Your Project

npm install lodash --save

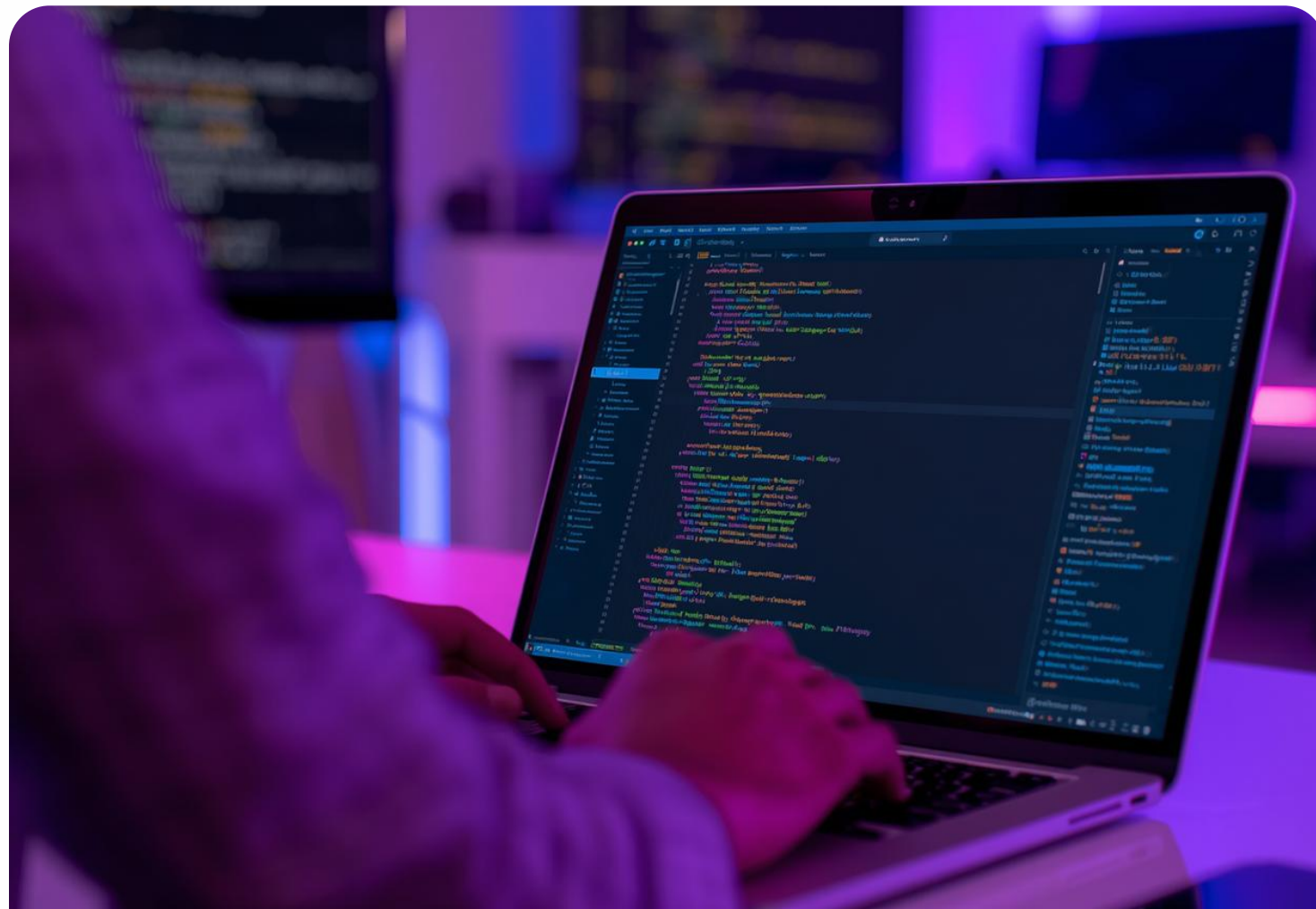
The `--save` flag adds the package to dependencies in package.json automatically.

node_modules Folder

NPM creates this folder to store all installed packages and their dependencies.

Example Usage

After installing, require the package in your code and use its functions immediately.



WHY -- SAVE IS CRITICAL

Dependency Management

The --save flag ensures packages are recorded in package.json. Never include node_modules in version control—add it to .gitignore instead. When others clone your project, running npm install automatically downloads all dependencies listed in package.json. Without --save, dependencies won't be tracked and won't install on other machines or deployment servers. Although, in the newer versions (npm 5+), --save isn't necessary to use.

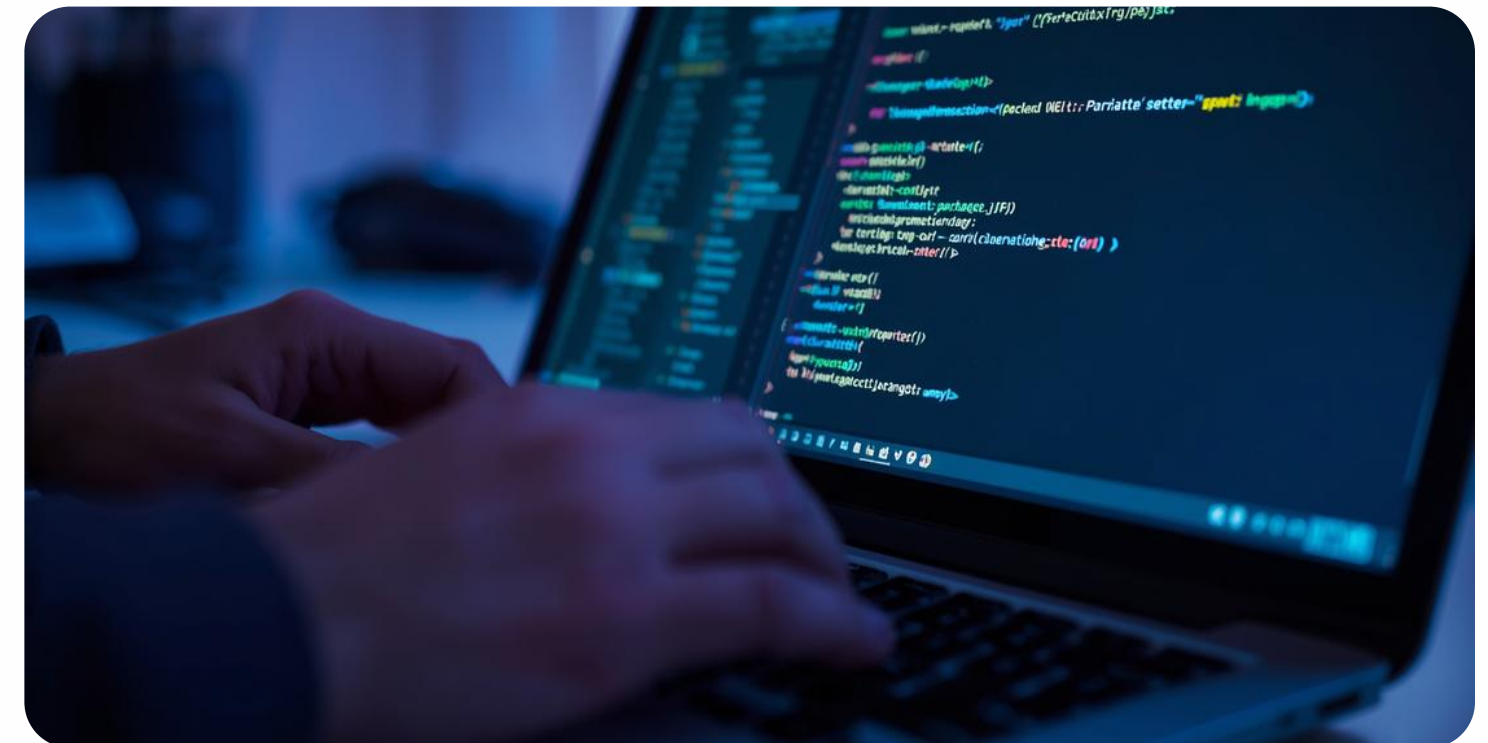




Dev Dependencies Explained

Dev dependencies are packages needed only during development, not in production. Examples include Gulp and Gulp-Sass for task running and compilation. Install with: `npm install gulp --save-dev`

Using `npm install --production` installs only regular dependencies, skipping dev dependencies entirely. This keeps production builds lightweight and efficient, excluding build tools from deployment.

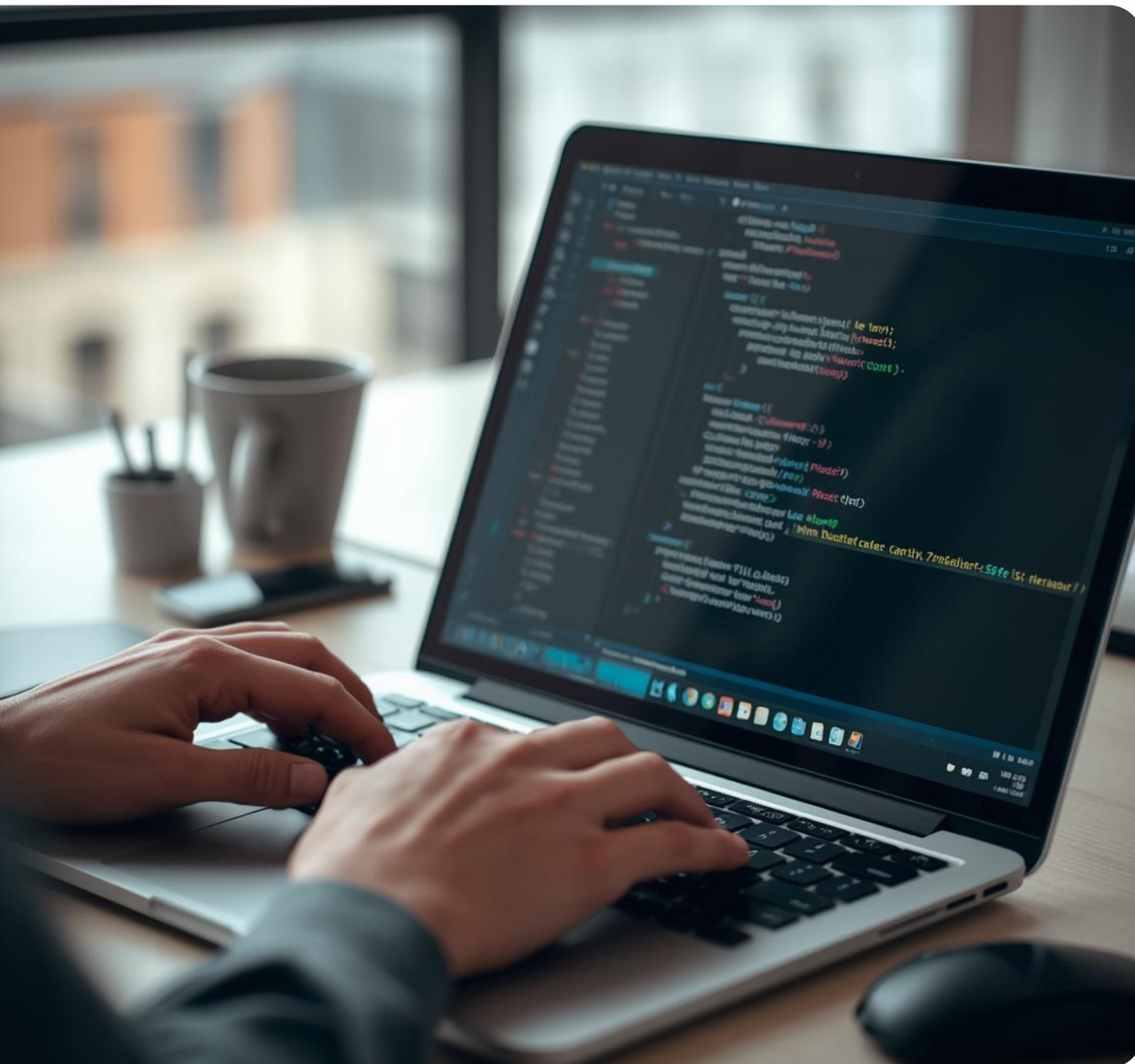


DEV DEPENDENCIES

UNINSTALLING PACKAGES

Clean Removal

Remove packages safely from your project using npm uninstall. Command: `npm uninstall gulp-sass --save-dev` removes the package from both node_modules folder and package.json. Multiple aliases available: `npm remove`, `npm rm`, `npm un`. The `--save-dev` flag ensures dev dependencies are properly removed from package.json.



Managing Package Versions

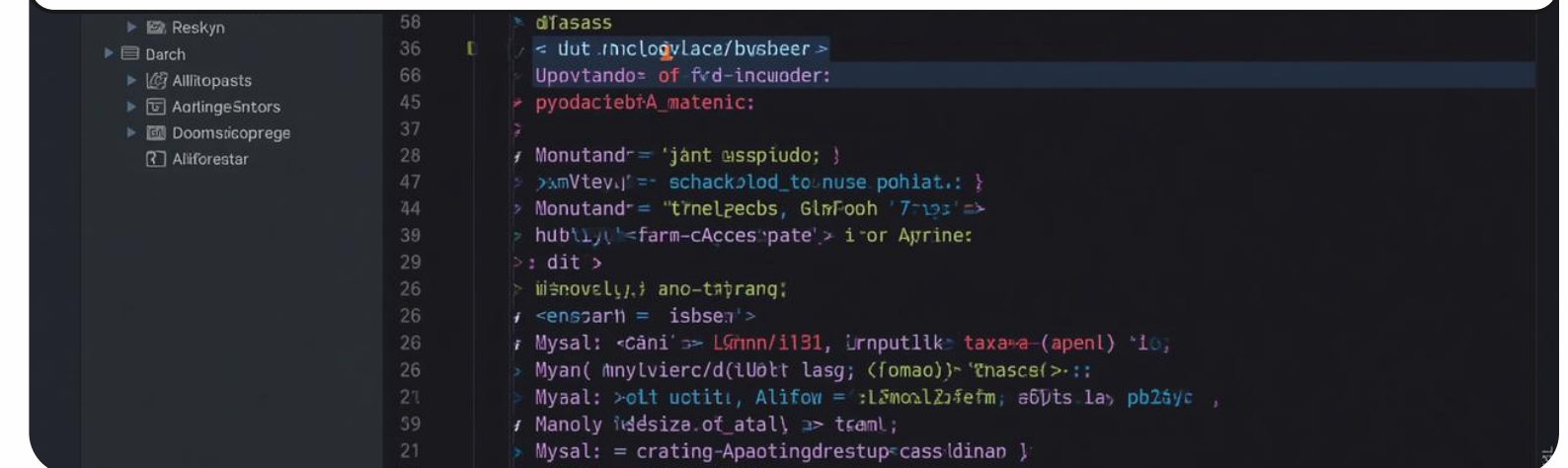
**Install Exact Version**

Use `npm install lodash@4.17.3 --save` to install a specific version of any package for precise control over your dependencies.

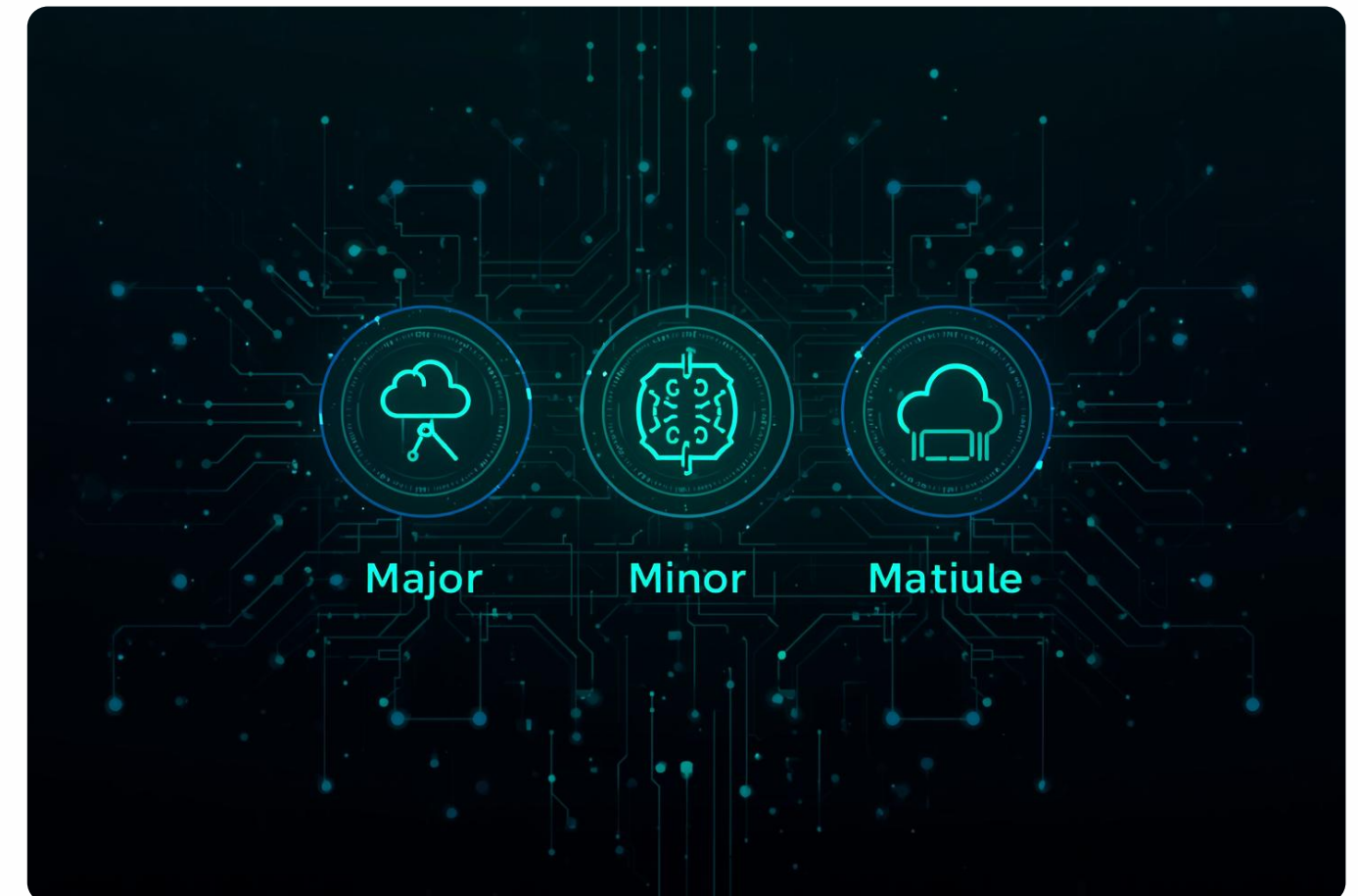
**Update Packages**

Run `npm update lodash` to update a package to its latest version within the allowed range specified in your `package.json` file.

SEMANTIC VERSIONING



SEMANTIC VERSIONING



Understanding SemVer Format

Patch Version

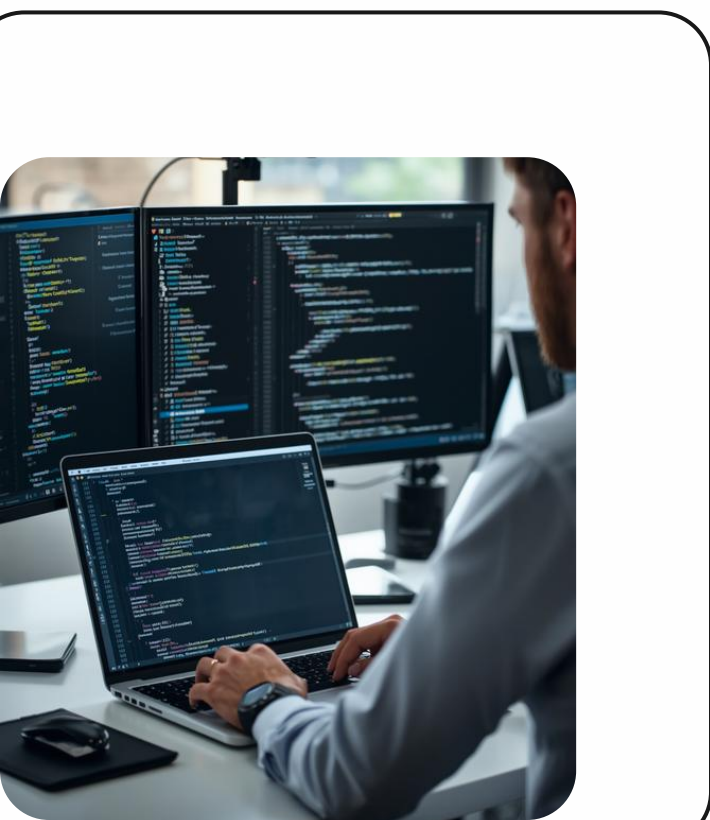
Bug fixes only, no breaking changes.
Example: 4.17.4 → 4.17.5. Safe to update automatically.

Minor Version

New features added, fully backward compatible. Example: 4.17.x → 4.18.x

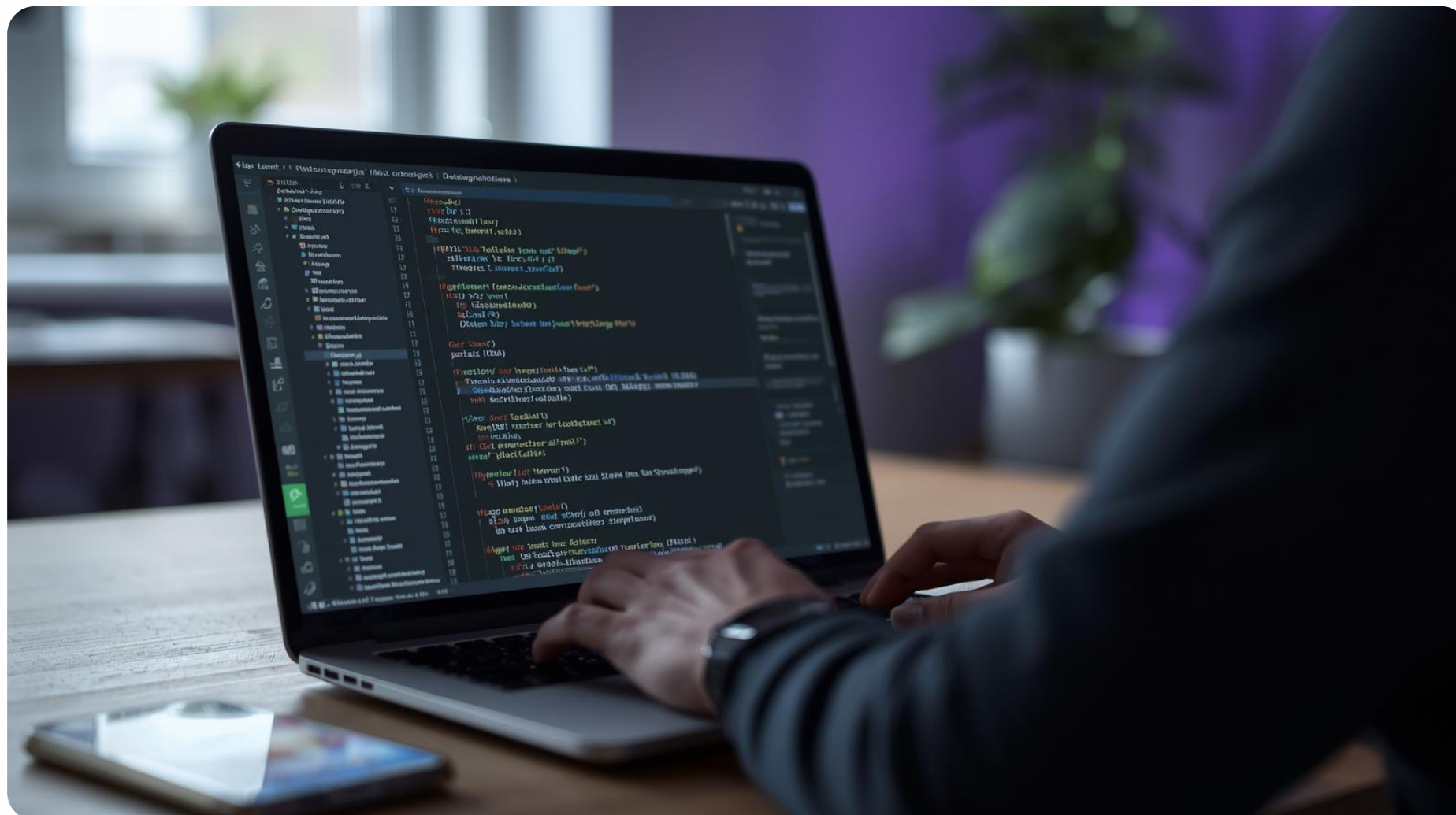
Major Version

Breaking changes that may require code updates. Example: 4.x → 5.x.
Review before updating.



VERSION SYMBOLS

How Prefixes Affect Installed Versions



Caret (^)

Installs latest
minor version
(`^4.17.4` → `4.x.x`)

Tilde (~)

Installs latest
patch version only
(`~4.17.4` → `4.17.x`)

No Symbol

Installs exact
version as
specified.

Asterisk (*)

Installs absolute
latest version (not
recommended).

GLOBAL PACKAGE S

Install Global

Use -g flag:
npm install nodemon -g
npm install live-server -g

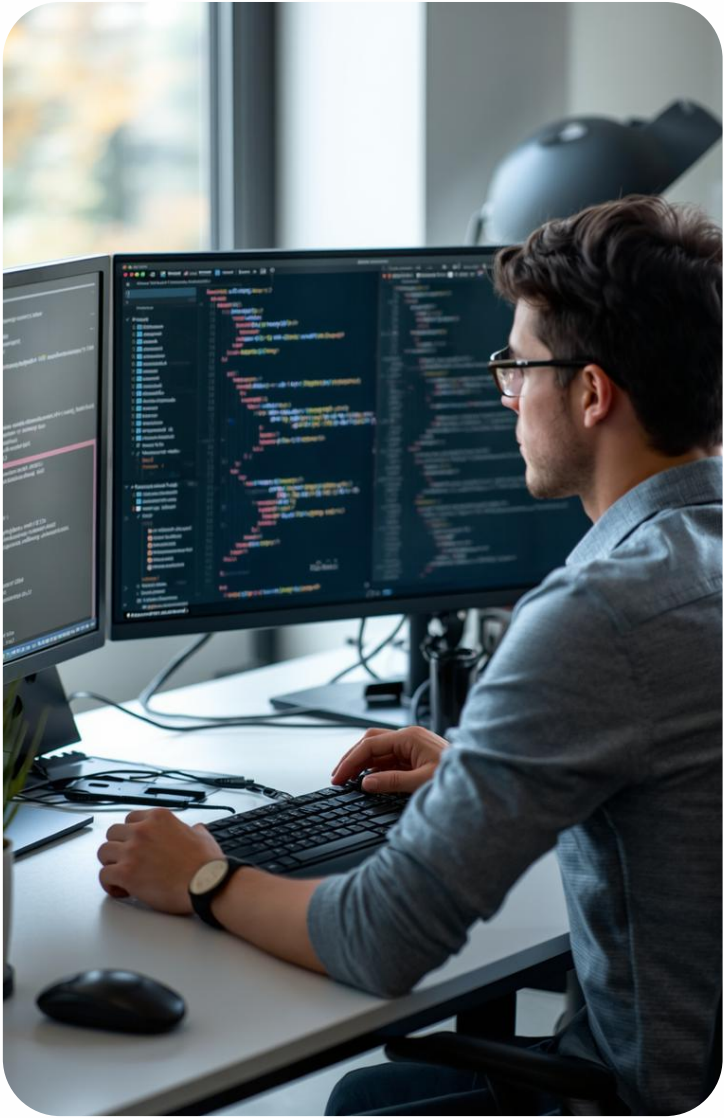
Key Difference

Not listed in package.json or project node_modules folder.

Find Location

npm root -g shows global install path.
Remove: npm uninstall nodemon -g

```
1  crananal  
2  Vtall_chalent-ag_};  
3  masteninstall-on-bentScribelscom/lcom,  
4  }  
5  Thintstier;; / ameg: (LecA*)  
6  //ff: dater wveflatediont:: ygvep,  
7  }  
8  //ff Accher mastiSaTional\{: Teverl,"  
9  facdyoic unndeSages ({}),  
10  femm_inlajSeralll sumjoct(,uno,";  
11  ff ff Nawallintal solor:-Phodd,  
12  }  
13  }  
14  }  
15  }
```

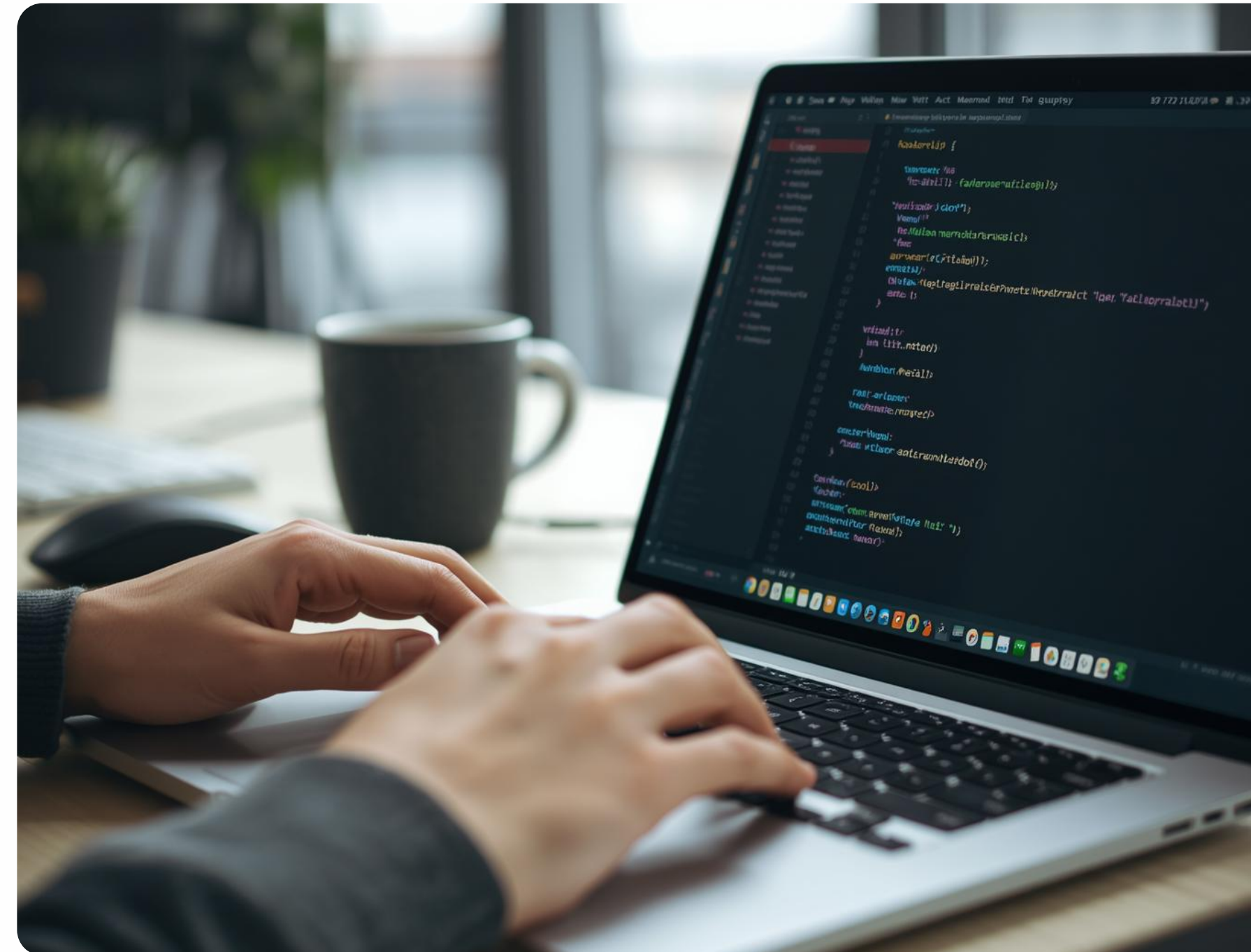




Development Helper

Nodemon watches your application files and automatically restarts the server whenever changes are detected. Instead of running `node app.js`, simply use `nodemon app.js`. This eliminates manual restarts during development, dramatically speeding up your workflow and letting you focus on writing code rather than managing processes.

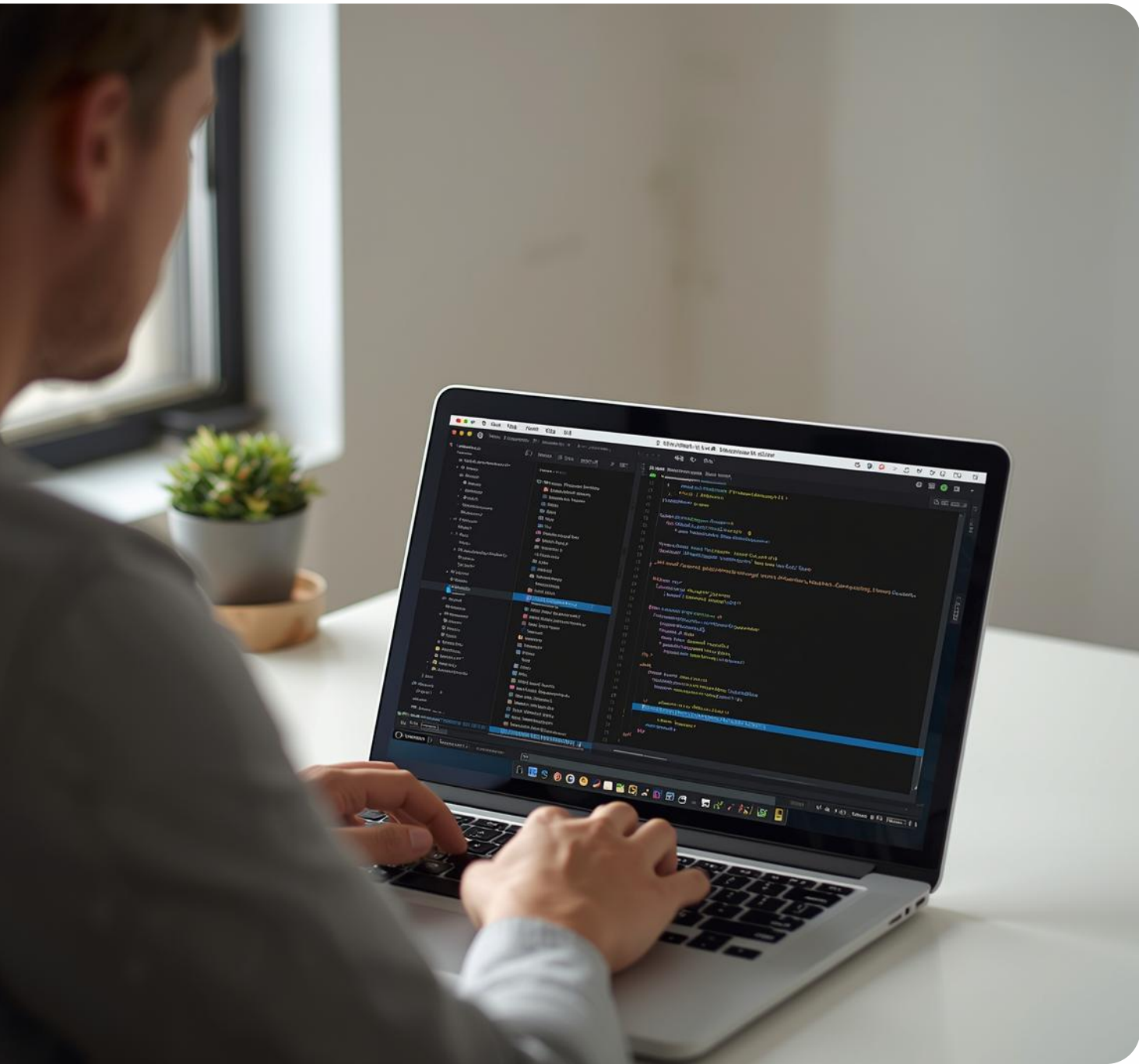
NODEMON



LIVE-SERVER - EASY LOCAL HOSTING

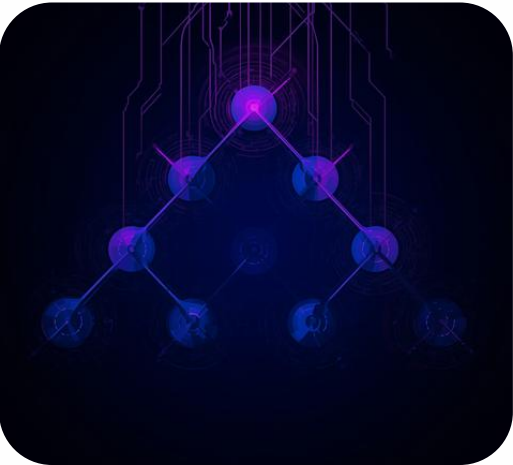
Quick
Frontend
Development

Live-server is a simple development server with live reload capability. Install globally with npm install live-server -g or as a dev dependency. Running live-server serves your current directory on localhost and automatically opens your default browser. Any file changes trigger instant browser refresh—perfect for frontend or static site development workflows.

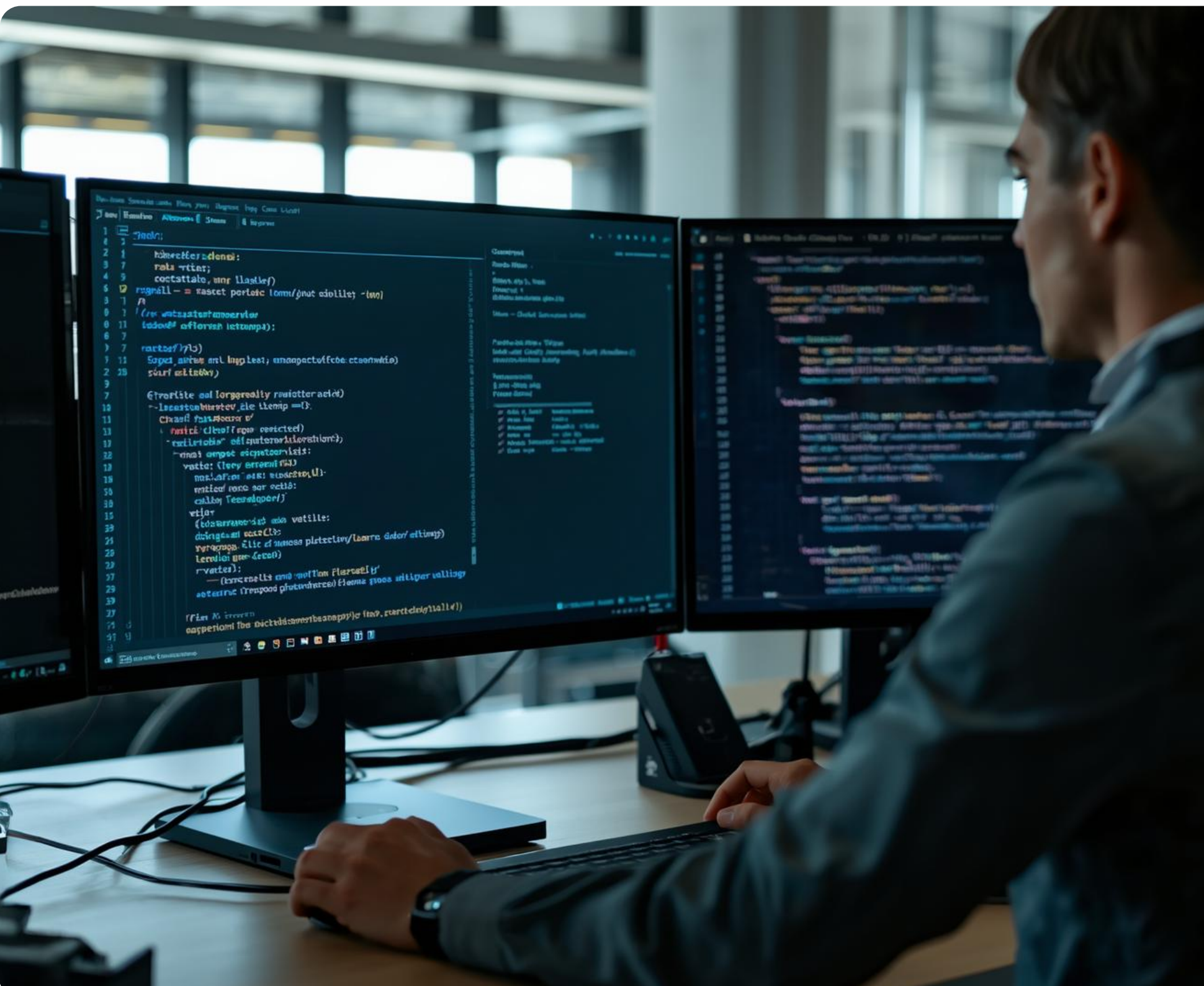


LISTING INSTALLED PACKAGES

Viewing Dependencies



Use npm list to view all installed packages and their dependency trees. This shows the complete hierarchy of packages in your project. npm list --depth=0 displays only top-level packages you directly installed, hiding nested dependencies for a cleaner view. npm list --depth=1 shows one level of dependencies, revealing immediate sub-dependencies of your packages. These commands help you understand what's installed and debug dependency issues quickly.



NPM SCRIPTS

Automating Common Tasks

Scripts Object

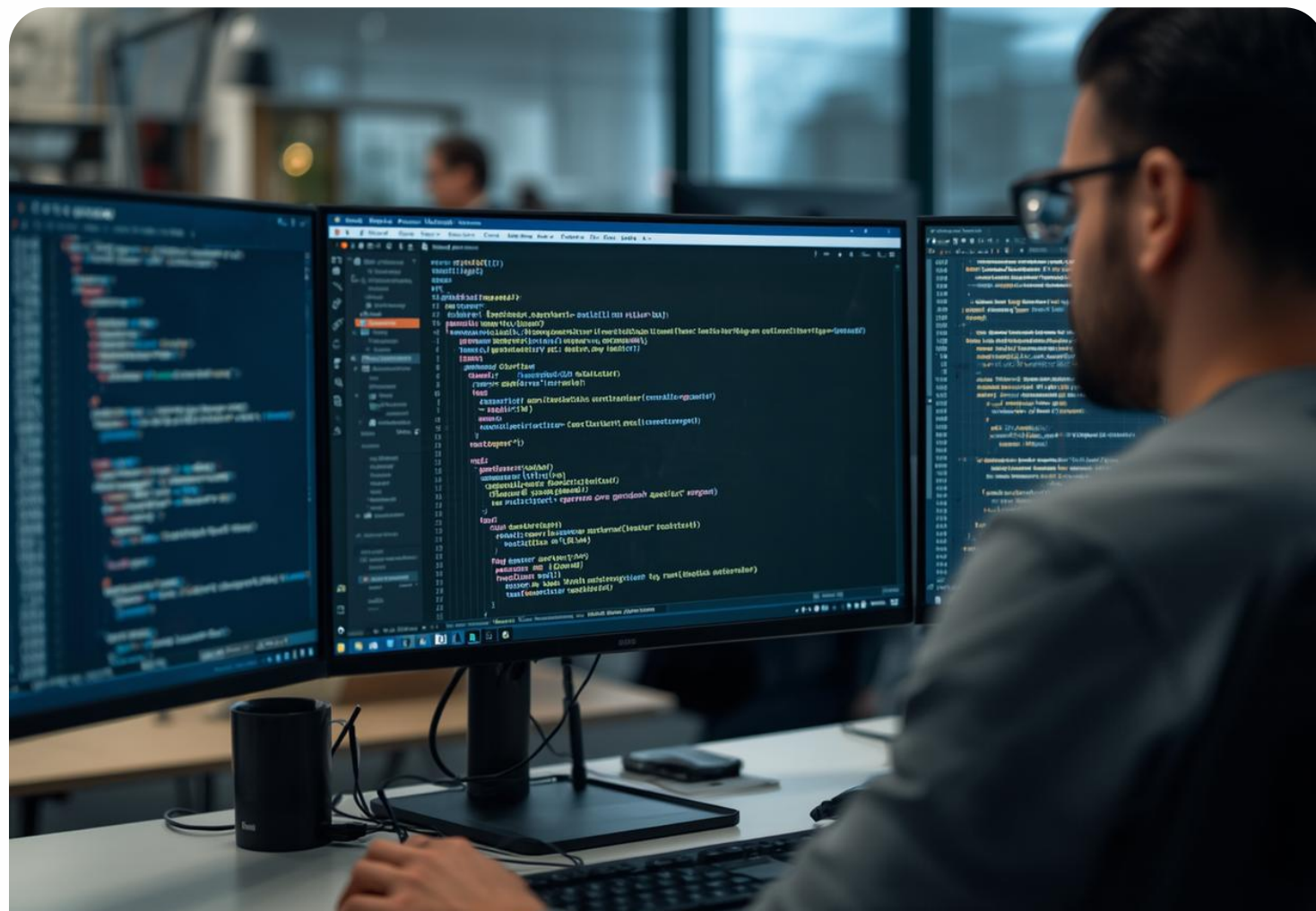
Scripts are defined in the "scripts" object within package.json.

Simplify Workflow

Scripts make complex commands simple and enable easier deployment.

Running Scripts

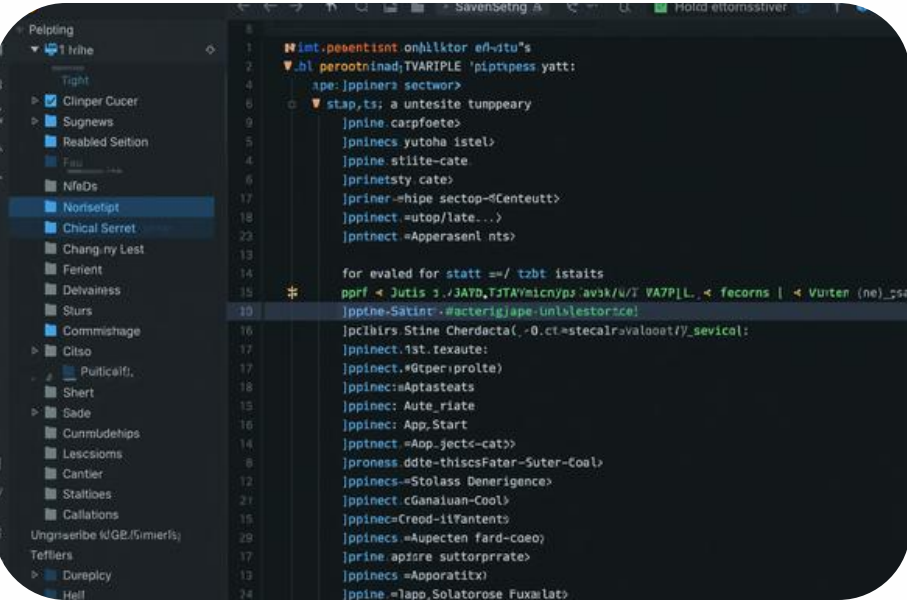
Run with `npm start` or `npm run [script-name]` for custom scripts.



EXAMPLE NPM SCRIPTS

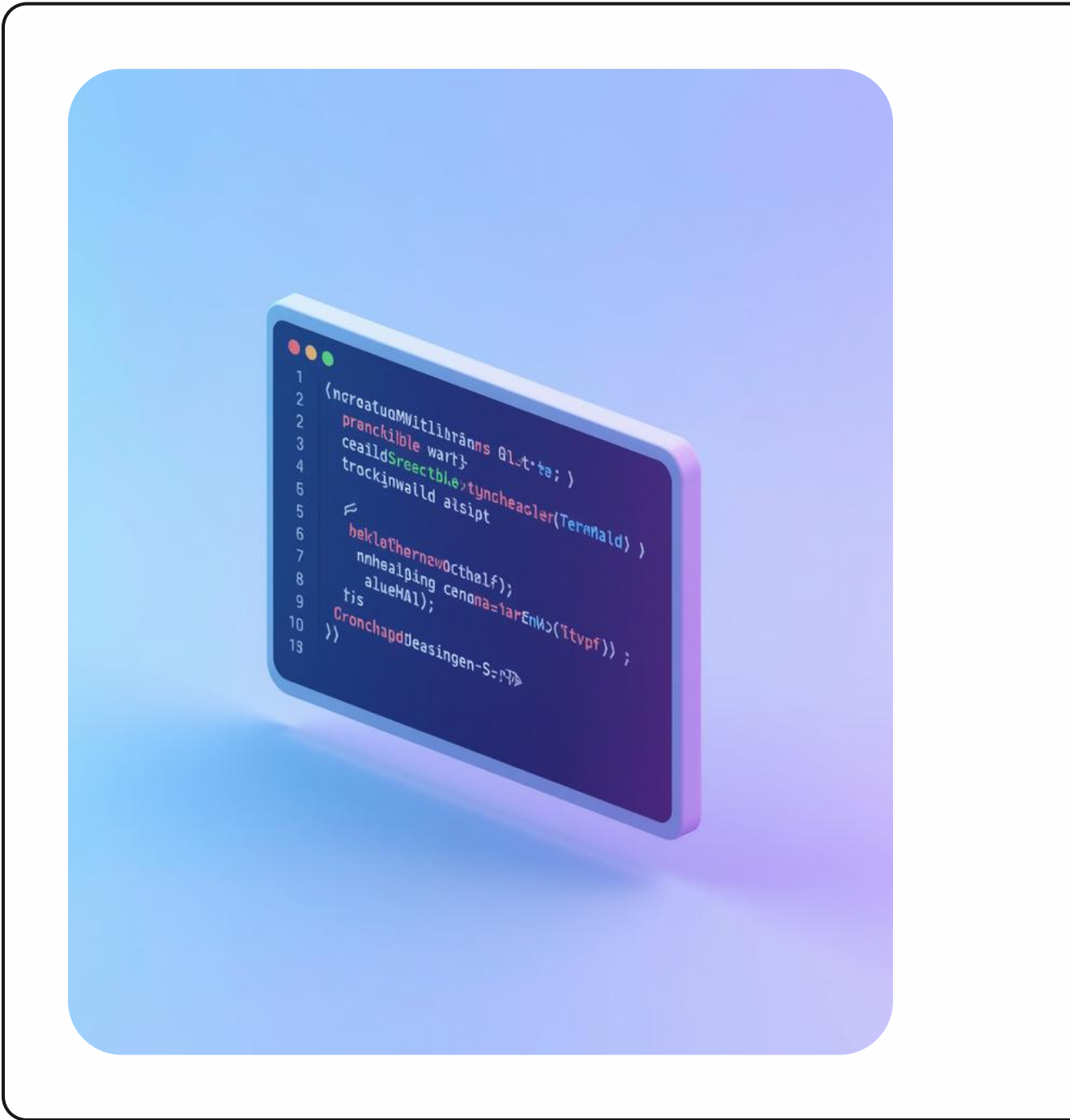
"start": "node app.js"

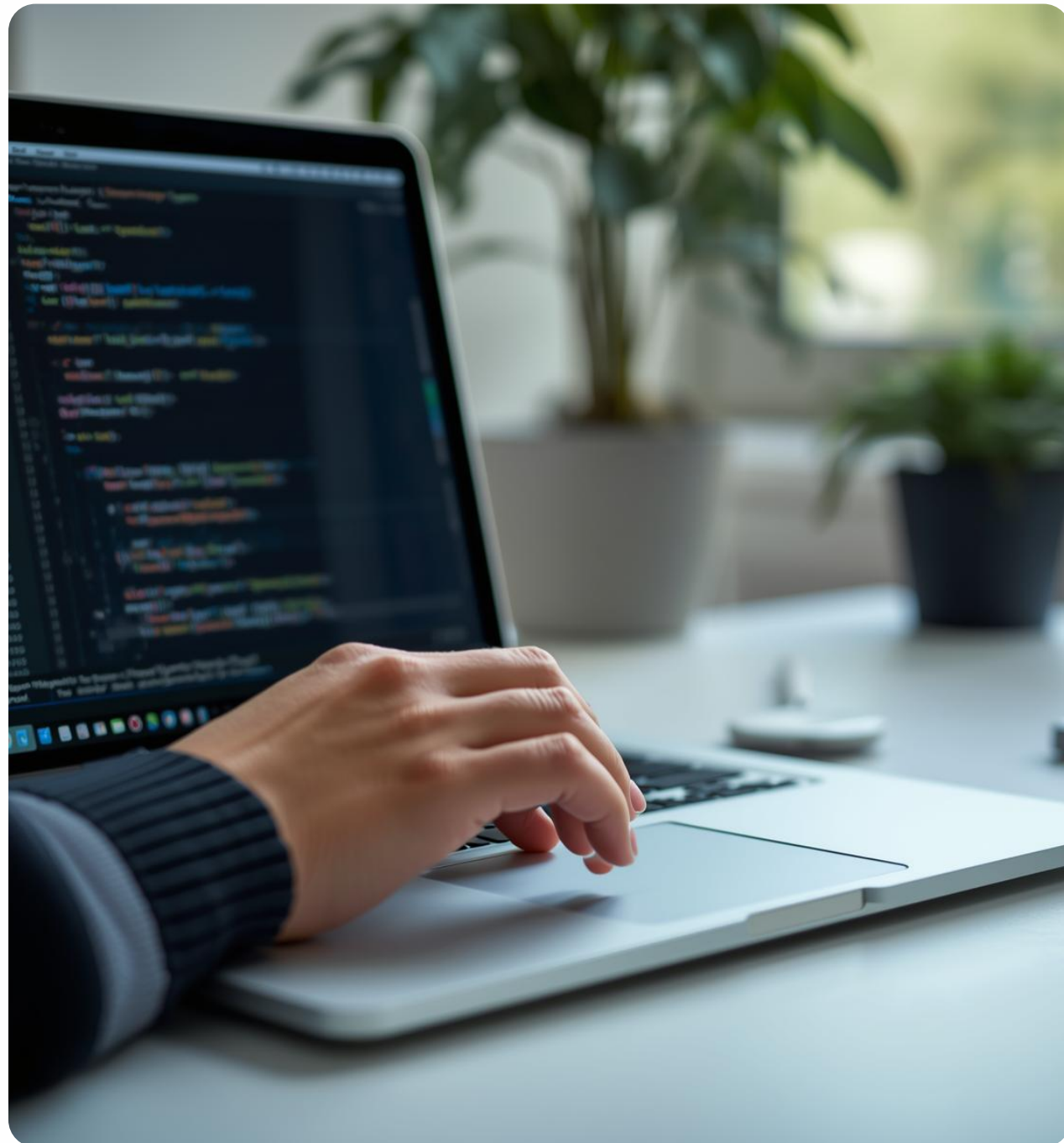
Launch your application with npm start. This is the default entry point for Node.js apps.



"server": "live-server"

Run a local development server with npm run server. Auto-refreshes on file changes.





Home

Commands

Scripts

Resources

NPM Best Practices & Key Takeaways

Save Flags

Always use `--save` or `--save-dev` flags when installing.

SemVer

Understand semantic versioning symbols.

Git Ignore

Add `node_modules` to `.gitignore` for version control.

NPM Scripts

Automate tasks using scripts in `package.json`.

BEST PRACTICES



THANK YOU!

Questions?



npmjs.com



docs.npmjs.com



npm@support.com



nodejs.org/download